Course INFO-H413: Heuristic Optimization

Implementation Exercise 1: Iterative Improvement and
Variable Neighborhood Descent for the Linear Ordering Problem

M'hamdi Zakaria (Matricule: 000630126)
Master in Computer Science

Professor: Thomas Stützle

April 2026

Abstract

This report presents the implementation and statistical analysis of iterative improvement algorithms and Variable Neighborhood Descent (VND) applied to the Linear Ordering Problem (LOP). Twelve algorithmic variants were evaluated by crossing two initialization strategies (random permutation and the Chenery–Watanabe heuristic), three neighborhood structures (Transpose, Exchange, and Insert), and two pivoting rules (First-Improvement and Best-Improvement). Delta-evaluations are systematically used to ensure computational efficiency. Statistical analysis using paired Wilcoxon signed-rank tests and Student's t-tests reveals that the CW initialization significantly outperforms random initialization, and that the Insert neighborhood under the First-Improvement rule yields the best solution quality. Two VND variants are then compared, confirming that neighborhood ordering has no statistically significant impact on final quality, although VND_TIE proves to be computationally more efficient and is therefore recommended.

Contents

1 Introduction **3**

2 Problem Description **3**

3 Material and Methods **3**
3.1 Initial Solution Generation . . . 3
3.2 Neighborhood Structures and Delta-Evaluations . . . 4
3.2.1 Transpose Neighborhood . . . 4
3.2.2 Exchange Neighborhood . . . 4
3.2.3 Insert Neighborhood . . . 4
3.3 Pivoting Rules . . . 5
3.4 Iterative Improvement Algorithm . . . 5
3.5 Variable Neighborhood Descent (VND) . . . 5
3.6 Experimental Setup . . . 5
3.7 Statistical Tests . . . 6
3.8 Low-Level Optimizations . . . 6

4 Results and Analysis: Exercise 1.1 **6**
4.1 Impact of Initialization . . . 7
4.2 Impact of Neighborhood Structure . . . 7
4.3 Impact of Pivoting Rule . . . 8

5 Results and Analysis: Exercise 1.2 – Variable Neighborhood Descent **8**

6 Conclusion **9**

1 Introduction

Combinatorial optimization problems arise in numerous domains, from logistics and scheduling to economics and bioinformatics. The central challenge is that the solution space grows exponentially with instance size, rendering exact methods intractable beyond modest scales. This motivates the study of *heuristic* algorithms, which trade optimality guarantees for practical efficiency.

This report focuses on the *Linear Ordering Problem* (LOP), a classic NP-hard combinatorial problem with applications in input-output economic analysis, ranking in sport tournaments, and seriation in archaeology. The goal of this exercise is to implement, evaluate, and compare several iterative improvement algorithms, combining different initialization methods, neighborhood structures, and pivoting rules, in a rigorous experimental framework.

The report is organized as follows. Section 2 defines the LOP formally. Section 3 describes all algorithmic components and the experimental setup. Section 4 presents and interprets results for Exercise 1.1, and Section 5 covers the VND analysis of Exercise 1.2. Section 6 summarizes the main findings.

2 Problem Description

Given an $n \times n$ weight matrix C where c_{ij} is the weight of the arc from node i to node j, the LOP asks for a permutation π of the indices $\{1, \ldots, n\}$ that maximizes the sum of weights of *forward-pointing* arcs, i.e., those arcs (i, j) such that i appears before j in π:

$$f(\pi) = \sum_{i=1}^{n-1} \sum_{j=i+1}^{n} c_{\pi(i)\pi(j)} \tag{1}$$

Geometrically, this is equivalent to simultaneously reordering the rows and columns of C to maximize the sum of all entries strictly above the main diagonal. Since the number of permutations is $n!$, exhaustive search is computationally infeasible even for moderate n, fully justifying the use of local search heuristics. The instances used in this study have sizes $n = 150$ and $n = 250$.

3 Material and Methods

3.1 Initial Solution Generation

Local search requires a starting point from which to explore the solution space. Two generation strategies were implemented and compared:

Uninformed Random Picking. A uniformly random permutation is generated using a linear congruential pseudo-random number generator (Park–Miller constants). The permutation is initialized in ascending order and iteratively shuffled by swapping the element at the current position with a randomly chosen element from the remaining unassigned ones. This guarantees a uniform distribution over all permutations and serves as the uninformed baseline.

Chenery and Watanabe (CW) Heuristic (Static Variant). To generate a high-quality initial solution, we implemented a static variant of the CW heuristic. It ranks all nodes globally based on their net attractiveness score, defined as the difference between their outgoing and incoming arc weights:

$$\text{score}(i) = \sum_{j \neq i} (c_{ij} - c_{ji}) \tag{2}$$

Nodes are then sorted in descending order of this score to form the initial permutation. This globally greedy approach captures the asymmetry and dominance of each node within the matrix, producing a significantly better starting solution than random generation while maintaining an $\mathcal{O}(n^2)$ initialization time.

3.2 Neighborhood Structures and Delta-Evaluations

A neighborhood $\mathcal{N}(\pi)$ is the set of solutions reachable from π by a single *move*. Evaluating a full permutation from scratch requires $\mathcal{O}(n^2)$ operations. To ensure scalability, all three neighborhoods use *delta-evaluations* (also called speed-ups): rather than recomputing $f(\pi')$ entirely, they compute only the cost variation $\Delta f = f(\pi') - f(\pi)$ induced by the move. A move is accepted if $\Delta f > 0$.

Complexity and Delta-Evaluation

While the naive computation of the objective function (the total sum of weights above the diagonal) requires $\mathcal{O}(n^2)$ operations, our implementation relies on localized delta-evaluations to navigate the search space efficiently. By computing only the difference in cost between the current solution and its neighbor, the complexity for the Transpose neighborhood is reduced to $\mathcal{O}(1)$, while for Exchange and Insert, it is maintained at $\mathcal{O}(n)$.

3.2.1 Transpose Neighborhood

A *transpose* move swaps two adjacent elements at positions i and $i+1$. Because only the relative order of $\pi(i)$ and $\pi(i+1)$ changes, all other pairwise contributions remain identical. The delta can be computed in constant time $\mathcal{O}(1)$:

$$\Delta f = c_{\pi(i+1)\pi(i)} - c_{\pi(i)\pi(i+1)} \tag{3}$$

This neighborhood has size $\mathcal{O}(n)$, making it extremely fast per step but severely limited in exploratory power.

3.2.2 Exchange Neighborhood

An *exchange* move swaps two arbitrary elements at positions $i < j$. The elements between positions $i+1$ and $j-1$ shift implicitly; their mutual order is unaffected, but their relationships with the two swapped elements change. The delta is:

$$\Delta f = c_{\pi(j)\pi(i)} - c_{\pi(i)\pi(j)} + \sum_{k=i+1}^{j-1} \left(c_{\pi(j)\pi(k)} + c_{\pi(k)\pi(i)} - c_{\pi(i)\pi(k)} - c_{\pi(k)\pi(j)} \right) \tag{4}$$

This requires $\mathcal{O}(n)$ per evaluation and the neighborhood has size $\mathcal{O}(n^2)$.

3.2.3 Insert Neighborhood

An *insert* move removes the element at position i and reinserts it at position j, shifting the elements in between by one position. Only the interactions between the moved element and the displaced sub-block need to be recomputed. For a forward insertion ($i < j$), the intermediate elements shift backward by one position:

$$\Delta f = \sum_{k=i+1}^{j} \left(c_{\pi(k)\pi(i)} - c_{\pi(i)\pi(k)} \right) \tag{5}$$

For a backward insertion ($i > j$), the intermediate elements shift forward:

$$\Delta f = \sum_{k=j}^{i-1} \left(c_{\pi(i)\pi(k)} - c_{\pi(k)\pi(i)} \right) \tag{6}$$

This evaluates in $\mathcal{O}(n)$ and the neighborhood has size $\mathcal{O}(n^2)$.

3.3 Pivoting Rules

Once a neighborhood is defined, a *pivoting rule* determines which neighbor to select:

- **First-Improvement:** The neighborhood is traversed in a fixed order and the first move yielding $\Delta f > 0$ is immediately applied. The search then restarts from the updated solution. This rule is efficient per iteration and can explore many improving moves, but the result depends on the traversal order.
- **Best-Improvement:** All moves in the neighborhood are evaluated and the one with the highest Δf is applied. This follows the steepest ascent direction but requires evaluating the full neighborhood at each step, incurring a higher computational cost per iteration.

3.4 Iterative Improvement Algorithm

The general iterative improvement algorithm is:

```
π := GenerateInitialSolution()
while π is not a local optimum do
choose a neighbour π' ∈ N(π) such that f(π') > f(π)
π := π'
```

The algorithm terminates when no improving neighbor exists, i.e., when the current solution is a local optimum with respect to the chosen neighborhood.

3.5 Variable Neighborhood Descent (VND)

A fundamental limitation of single-neighborhood local search is that a local optimum for one topology may not be locally optimal for another. The Variable Neighborhood Descent algorithm systematically exploits this by cycling through a sequence of k neighborhoods $\mathcal{N}_1, \mathcal{N}_2, \mathcal{N}_3$. Whenever an improvement is found in the current neighborhood, the search restarts from $\mathcal{N}_1$; if no improvement is found, the search advances to the next neighborhood. The algorithm terminates when a simultaneous local optimum for all neighborhoods is reached.

The VND uses the First-Improvement rule and the CW initialization. Two orderings were evaluated:

1. **VND_TEI:** Transpose → Exchange → Insert
2. **VND_TIE:** Transpose → Insert → Exchange

3.6 Experimental Setup

All algorithms were implemented in C, compiled with GCC using `-O3` optimization, and executed on a machine equipped with an Intel Core i5-14600KF processor (14 cores, up to 5.3 GHz) and 32 GB DDR5 RAM under a Linux environment (WSL). Computation times were measured using standard C time libraries, excluding file I/O.

To ensure reproducibility across algorithms and fair comparison, the pseudo-random seed was fixed to the same constant value for each instance (set as the sum of all matrix elements). Each

algorithm was run once per instance. The benchmark set consists of 78 instances with sizes 150 and 250.

The percentage deviation from the best-known solution for instance i and algorithm k is defined as:

$$\Delta_{ki} = 100 \times \frac{f_{\text{best},i} - f_{ki}}{f_{\text{best},i}} \tag{7}$$

where $f_{\text{best},i}$ is the best-known objective value for instance i and f_{ki} is the value found by algorithm k.

3.7 Statistical Tests

To determine whether observed differences in solution quality are statistically significant, two paired hypothesis tests were applied at significance level $\alpha = 0.05$:

- **Paired Student's t-test:** Tests whether the mean difference between two algorithms is zero, assuming approximately normally distributed differences.
- **Wilcoxon signed-rank test:** A non-parametric alternative that tests whether the median of the pairwise differences is zero. It is more robust when the normality assumption does not hold, as is common with heuristic performance data.

Both tests exploit the paired structure of the data: since both algorithms are evaluated on the same instances, inter-instance variability is naturally controlled for. All tests were computed using the R software environment.

3.8 Low-Level Optimizations

To minimize the overhead of local search, delta-evaluation functions (`compute_exchange_delta` and `compute_insert_delta`) are declared as `static inline`. This provides two critical advantages:

- **Function Inlining:** Combined with the `-O3` compiler flag, it suggests the compiler to replace function calls with the actual code body. This eliminates the assembly-level overhead of jumping to a function and managing the stack frame for every neighbor evaluation (which occurs millions of times).
- **Scope Optimization:** The `static` keyword limits the visibility of these functions to the translation unit, allowing the compiler to perform more aggressive inter-procedural optimizations within `optimization.c`.

These optimizations ensure that the theoretical complexities ($\mathcal{O}(1)$ for Transpose and $\mathcal{O}(n)$ for Exchange/Insert) translate into maximum practical throughput.

4 Results and Analysis: Exercise 1.1

Table 1 summarizes the performance of all 12 iterative improvement algorithms: the average percentage deviation from the best-known solution, the standard deviation of this deviation across instances, and the total computation time summed over all instances.

Table 2 reports the paired statistical tests designed to isolate the contribution of each algorithmic component.

Table 1: Performance of the 12 iterative improvement algorithms: average deviation, standard deviation, and total execution time across all instances.

Initialization	**Neighborhood (Pivot)**	**Avg. Dev. (%)**	**Std. Dev. (%)**	**Total Time (s)**
CW	Insert (First)	1.691	0.360	113.31
CW	Insert (Best)	1.841	0.320	27.13
CW	Exchange (First)	2.407	0.439	150.70
CW	Exchange (Best)	2.957	0.427	24.44
CW	Transpose (Best)	15.735	1.430	< 0.01
CW	Transpose (First)	15.864	1.420	< 0.01
Random	Insert (First)	2.021	0.451	154.42
Random	Insert (Best)	2.304	0.464	31.04
Random	Exchange (First)	2.816	0.481	180.97
Random	Exchange (Best)	3.603	0.505	27.65
Random	Transpose (Best)	34.470	3.803	< 0.01
Random	Transpose (First)	34.606	3.798	< 0.01

Table 2: Summary of paired statistical tests ($\alpha = 0.05$). Results from both the t-test and the Wilcoxon signed-rank test are reported.

Component tested	**Compared variants**	**t-test p-value**	**Wilcoxon p-value**	**Significa**
Initialization	CW vs. Random (all algs.)	$< 2.2 \times 10^{-16}$	$< 2.2 \times 10^{-16}$	Yes
Pivot Rule	Insert_First vs. Insert_Best (CW)	8.74×10^{-4}	6.82×10^{-4}	Yes

4.1 Impact of Initialization

The CW heuristic consistently and substantially outperforms random initialization, reducing the average deviation by 6.55 percentage points across all paired variants. Both the t-test ($t = -16.00$, $df = 467$) and the Wilcoxon test ($p < 2.2 \times 10^{-16}$) confirm this advantage is highly statistically significant. This result is intuitive: CW is a constructive greedy method that maximizes local attractiveness, placing the starting solution in a region of the search space much closer to good local optima than a uniform random permutation, accelerating subsequent local search convergence.

However, the performance gain from CW initialization is heavily topology-dependent. The 6.55 percentage point average reduction is severely skewed by the weak Transpose neighborhood (where CW reduces deviation by ~ 18.7 points). When utilizing the powerful Insert topology, the gap between CW and Random initializations is drastically reduced to 0.33 percentage points (1.691% vs. 2.021%). This demonstrates that while a sufficiently strong local search operator largely compensates for an uninformed starting state, the structural advantage of the CW initialization remains small but consistent. The exploratory depth of the Insert neighborhood mitigates, but does not entirely eliminate, the penalty of a poor initial permutation.

4.2 Impact of Neighborhood Structure

The choice of neighborhood is the most impactful design decision. The results clearly stratify into three groups. The Transpose neighborhood performs catastrophically, reaching average deviations of 15.73% (CW) and 34.47% (Random). The Exchange neighborhood achieves moderate quality (2.41–3.60%). The Insert neighborhood is the clear winner (1.69–2.30%).

This ordering is well explained by the theoretical properties of each neighborhood. Transpose only swaps adjacent elements, giving it a neighborhood size of $\mathcal{O}(n)$ and making only micro-

adjustments. The algorithm immediately gets trapped in very shallow local optima with extremely short computation times (below 10 ms total). Exchange has a larger neighborhood ($\mathcal{O}(n^2)$ moves) but swapping distant elements heavily disrupts the relative order of all intermediate elements, potentially destroying good partial sequences. Insert, by contrast, relocates a single element to an arbitrary position while preserving the relative order of all other elements; this is structurally analogous to insertion sort and makes it perfectly suited to the LOP, where the objective is inherently an ordering criterion.

4.3 Impact of Pivoting Rule

Comparing First-Improvement and Best-Improvement within the Insert neighborhood using CW initialization reveals a counterintuitive but well-known phenomenon in heuristic optimization: First-Improvement finds significantly better solutions despite being conceptually "less greedy" than Best-Improvement. Concretely, CW+Insert+First achieves 1.691% average deviation versus 1.841% for CW+Insert+Best. Both the paired t-test ($p = 8.74 \times 10^{-4}$) and Wilcoxon test ($p = 6.82 \times 10^{-4}$) confirm this difference is statistically significant.

The explanation lies in the convergence dynamics. Best-Improvement evaluates the entire neighborhood at each step and always climbs the steepest local gradient, which causes rapid convergence to the nearest local optimum in roughly 27 seconds. First-Improvement applies the first valid move encountered and restarts immediately, following a much longer and more erratic trajectory through the search space (over 113 seconds). This effectively acts as a natural diversification mechanism: rather than immediately ascending the steepest slope, the algorithm explores a richer variety of improving directions and discovers deeper basins of attraction. The total execution time is approximately 4 times higher for First-Improvement, representing the cost of this more thorough exploration.

This result has important practical implications: the faster-converging Best-Improvement strategy does not necessarily find better solutions, and allocating more computation time via First-Improvement pays off in solution quality.

5 Results and Analysis: Exercise 1.2 – Variable Neighborhood Descent

Table 3 presents the performance of the two VND variants, and Table 4 reports the associated statistical test.

Table 3: Performance of the two VND variants using CW initialization and First-Improvement.

Algorithm (order)	**Avg. Dev. (%)**	**Std. Dev. (%)**	**Total Time (s)**
VND_TEI (Transpose → Exchange → Insert)	1.766	0.370	156.61
VND_TIE (Transpose → Insert → Exchange)	1.704	0.352	108.12

Table 4: Wilcoxon and t-test comparison of the two VND orderings.

Component	**Variants compared**	**t-test p**	**Wilcoxon p**	**Significant?**
Exploration order	VND_TEI vs. VND_TIE	0.1122	0.1325	No

Both VND variants achieve competitive performance. VND_TIE reaches 1.704% average deviation and VND_TEI reaches 1.766%. However, neither variant outperforms the best standalone algorithm: CW+Insert+First remains the best overall configuration at 1.691%. This indicates

that for this specific configuration, the VND framework does not provide an additional quality gain. The Insert neighborhood alone is already sufficiently powerful to reach high-quality local optima. Forcing transitions through the weaker Transpose and heavier Exchange neighborhoods merely disrupts the search trajectory and increases computational overhead without improving the final objective value.

Regarding the comparison between orderings, both the t-test ($p = 0.1122$) and the Wilcoxon test ($p = 0.1325$) fail to reject the null hypothesis of no difference at $\alpha = 0.05$. We therefore conclude that the *specific ordering* of neighborhoods does not significantly affect the final solution quality, provided all three neighborhoods are included.

This result is scientifically coherent. The VND mechanism ensures that, regardless of ordering, the algorithm eventually exercises all three neighborhoods before terminating. The Insert neighborhood, which provides the most powerful improvements, is reached in both variants. The intermediate neighborhoods (Transpose and Exchange) serve primarily to escape from local optima that the dominant neighborhood cannot resolve, and their relative order has little bearing on the final result.

However, VND_TIE enjoys a computational advantage: its total execution time is 108.12 seconds versus 156.61 seconds for VND_TEI, a reduction of approximately 31%. By placing Insert before Exchange in the sequence, VND_TIE reaches the most effective neighborhood sooner in each cycle, reduces the number of calls to the more expensive Exchange neighborhood, and converges faster overall.

Furthermore, the overall execution efficiency was significantly enhanced by the integration of incremental updates. By avoiding full cost recalculations within the nested local search loops, the throughput of evaluated neighbors per second was maximized. This algorithmic choice, combined with the aggressive `-O3` compiler optimizations, allowed the VND algorithms to converge in negligible time even on the largest instances ($n = 250$). Consequently, this makes VND_TIE the practically preferred configuration: statistically equivalent in quality but significantly faster while providing a robust balance between solution quality and computational effort.

6 Conclusion

This study evaluated twelve iterative improvement algorithms and two VND variants on a benchmark set of LOP instances, combining rigorous implementation with formal statistical testing. The main conclusions are:

The **neighborhood structure** is the single most decisive factor: Insert far outperforms Exchange and Transpose, owing to its property of preserving the relative order of all non-moved elements while relocating one element to a position that is locally optimal within the current sequence. Transpose's severe limitations confirm that neighborhood size and structural relevance to the problem are critical design criteria.

The **pivoting rule** matters significantly: First-Improvement produces better final solutions than Best-Improvement by following a longer, more exploratory search trajectory at the cost of increased computation time. This counter-intuitive result is a well-known phenomenon in heuristic optimization and underscores the dangers of premature convergence.

The **initialization strategy** also has a measurable and statistically significant impact: the CW heuristic provides a consistently better starting point than random generation, reducing average deviation by over 6.5 percentage points. However, this effect is largely driven by the weakest neighborhood (Transpose), and the performance gap narrows to a negligible margin once a powerful local search operator like Insert is applied.

Finally, the **VND** framework does not strictly improve over the best single-neighborhood search in this context, as the Insert operator is already sufficiently dominant. However, among the VND configurations, the ordering of neighborhoods does not significantly affect solution quality, making VND_TIE (Transpose $\rightarrow$ Insert $\rightarrow$ Exchange) the recommended configuration due to its superior computational efficiency.